

Credit Card Approval Prediction

Project Documentation

A machine-learning web application that predicts whether a credit card application will be approved or rejected, based on real Indian credit-underwriting factors (CIBIL score, income, debt-to-income ratio, employment history) and RBI/bank-style eligibility rules.

1. Project Overview

Banks and financial institutions receive a high volume of credit card applications daily, many of which are rejected due to insufficient income, poor credit history, or excessive existing debt. This project automates that screening decision using machine learning, trained on applicant financial and demographic data, and deployed as a live web application.

- Web-based application built with Python and Flask
- Four classification algorithms trained and compared: Logistic Regression, Decision Tree, Random Forest, XGBoost
- Hard eligibility rules (age, minimum income, minimum CIBIL score) applied before ML scoring, matching real Indian bank policy
- Full user system with accounts, authentication, and prediction history
- Relational database (SQLite) matching the project's Entity-Relationship design

2. Entity-Relationship Design

The database schema below matches the project's ER diagram: Users manage Applicant_Details records; each applicant has associated Credit_History and receives one Approval_Prediction, generated by one of the trained ML_Model entries.

Entity	Key Fields	Relationship
Users	UserID (PK), Name, Email, Password, Role	1 user -> many Applicant_Details
Applicant_Details	ApplicantID (PK), UserID (FK), Age, Gender, Income, CreditScore, ...	1 applicant -> many Credit_History; 1:1 Approval_Prediction
Credit_History	HistoryID (PK), ApplicantID (FK), MonthsBalance, PaymentStatus	Reference data used in model training
ML_Model	ModelID (PK), ModelName, AlgorithmType, Accuracy, ModelFile	1 model -> many Approval_Prediction
Approval_Prediction	PredictionID (PK), ApplicantID (FK), ModelID (FK), ApprovalResult, RiskScore	Result of each prediction

3. Technical Architecture

The application follows a layered architecture:

- User Layer: web browser access (credit analyst or applicant)
- Presentation Layer: Flask-rendered HTML templates (home, application form, prediction result, history, auth pages)
- Application Layer: Flask route handling, form validation, data preprocessing (scaling/encoding), model loading and inference

- Machine Learning Layer: four trained classifiers, evaluated by accuracy/ROC-AUC/confusion matrix, best model selected automatically
- Data Layer: raw applicant + payment-history data, cleaned, feature-engineered, and split 80/20 for training
- Deployment Layer: GitHub for version control, Render for live public hosting; IBM Watson Machine Learning deployment script included and tested locally (see Section 6)

4. Dataset & Feature Engineering

Two tables mirror the structure of real bureau data: `application_record` (applicant profile) and `credit_record` (monthly payment history). The multi-class payment STATUS history is converted into a binary label: any month 60+ days past due marks the applicant high-risk.

Features used

- CIBIL credit score (300-900)
- Annual income (INR) and debt-to-income ratio
- Employment status and length
- Age, gender, education level, family status, housing type
- Number of existing credit accounts

5. Model Training & Evaluation

All four models were trained on an 80/20 stratified train-test split and evaluated on accuracy, ROC-AUC, confusion matrix, and classification report. The best model by ROC-AUC is automatically selected and deployed.

Model	Accuracy	ROC-AUC	Status
Logistic Regression	78.7%	0.760	
Decision Tree	78.2%	0.711	
Random Forest	80.7%	0.763	
XGBoost	79.8%	0.763	Deployed (best ROC-AUC)

6. Deployment

6.1 Live Application (Render)

The application is deployed and publicly accessible on Render, a cloud hosting platform, connected directly to the project's GitHub repository for continuous deployment on every push.

6.2 IBM Watson Machine Learning

The project's architecture specifies IBM Cloud / Watson Machine Learning as the model-hosting layer. A deployment script (`deploy_to_watson_ml.py`) implements this using IBM's current `watsonx.ai` SDK: it registers the trained model with the Watson ML repository, creates a live scoring endpoint, and validates it with a test prediction. Its encoding/scaling logic has been verified locally against the production model. Live deployment to IBM Cloud requires credit/debit card identity verification (an IBM Cloud account requirement), which was not available at submission time; the script is complete and ready to run once account access is available.

7. Eligibility Rules (India)

Before any ML scoring, every application passes a hard eligibility gate reflecting common Indian bank policy:

- Minimum age 18 (no card can be issued to a minor)

- Applicants aged 18-20 without independent income are directed to a secured/add-on card instead
- Minimum annual income by employment type (e.g. ~Rs. 1,80,000/year for salaried applicants)
- Minimum CIBIL score of 650 (750+ preferred)
- Maximum age caps: 60 (salaried), 65 (self-employed)

8. Tech Stack

- Language: Python 3.12
- Web framework: Flask, Gunicorn (production server)
- ML/Data: scikit-learn, XGBoost, pandas, NumPy, Matplotlib, Seaborn
- Database: SQLite
- Frontend: HTML/CSS (Jinja2 templates)
- Deployment: GitHub, Render; IBM watsonx.ai SDK (prepared)

9. How to Run Locally

See README.md in the repository root for full setup instructions: virtual environment creation, dependency installation, dataset generation, model training, and running the Flask app.